

This keyword

This keyword

this keyword depends on how a function is invoked

this can refer to the global object.

In global

```
window.alert("123");
this.alert("456");
alert("789");
console.log(this===window); // true
```

```
let name="John";
console.log(this.name); //John
```

In a function

```
var firstName = "John";
var lastName = "Doe";
function fullName() {
    console.log(this.firstName +
                    " " + this.lastName);
}
fullName(); //John Doe
```

This keyword

In an object function , this refers to the object.

```
const person = {
  firstName: "John",
  lastName : "Doe",
  id       : 5566,
  fullName : function() {
    return this.firstName + " " + this.lastName;
  }
};
```

```
const person = {
  firstName  : "John",
  lastName   : "Doe",
  id         : 5566,
  myFunction : function() {
    return this;
  }
};
```

```
function getThis() {
  return this;
}

const obj1 = { name: "obj1" };
const obj2 = { name: "obj2" };

obj1.getThis = getThis;
obj2.getThis = getThis;

console.log(obj1.getThis());
// { name: 'obj1', getThis:
[Function: getThis] }
console.log(obj2.getThis());
```

This keyword

In an event, this refers to the element that received the event.

```
<button id="hidebtn">Remove Me!</button>
```

```
document.getElementById("hidebtn").addEventListener('click',hideMe);

function hideMe(){
    this.style.display='none';
}
```

This keyword - Explicit Function Binding

call() - invokes the function immediately with the specified this value

```
const person1 = {
  fullName: function() {
    return this.firstName + " "
              + this.lastName;
  }
}

const person2 = {
  firstName:"John",
  lastName: "Doe",
}

person1.fullName.call(person2);
// Return "John Doe":
```

person1.fullName calls person2 as an argument, **this** refers to person2, even if fullName is a method of person1:

This keyword - Explicit Function Binding

bind() - returns a new function with the this value permanently

```
const person1 = {
  firstName:"John",
  lastName: "Doe",
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
const person2 = {
  firstName:"Hege",
  lastName: "Nilsen",
}
//bind person2 to fullName() of person1
let fullName = person1.fullName.bind(person2);
// full name of member "Hege Nilsen"
```



Set Time

setTimeout

The setTimeout() method calls a function after a number of milliseconds.

```
setTimeout(function, milliseconds);
```

```
setTimeout(() =>{
    document.getElementById("greet").innerText = "Welcome to our page";
}, 2000);
```

Welcome to our page

Wait 2000 ms and then show message

```
<div id="count"></div>
<script>
    let c = 0;
    count();
    function count() {
        document.getElementById("count").innerText = c;
        c++;
        setTimeout(count, 1000);
    }
</script>
```



0
1
2
3
...

`clearTimeout()` method to prevent the function from starting

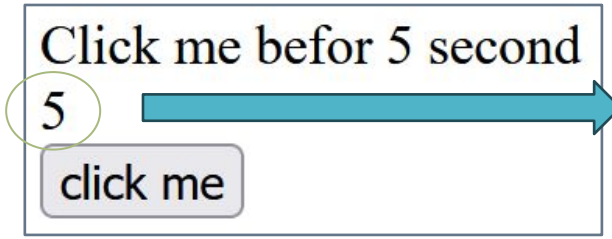
clearTimeout(`functionId`)

To clear a timeout, use the **id** returned from setTimeout():

```
myTimeout = setTimeout(function, milliseconds);
```

Then stop the execution by calling clearTimeout():

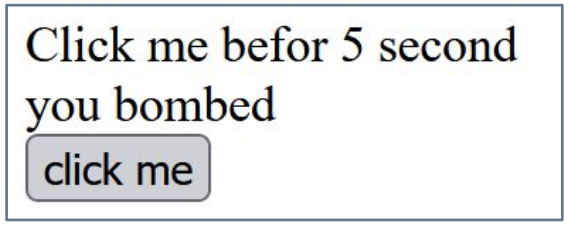
```
clearTimeout(myTimeout);
```



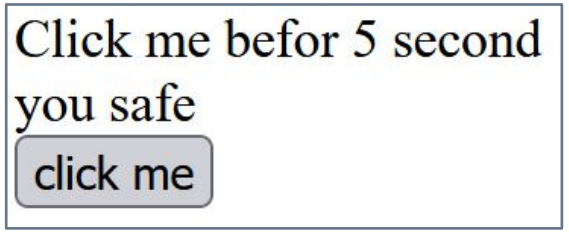
Counter count down

4 -> 3 -> 2 -> 1 -> 0

No click



Click intime




```
<div>Click me befor 5 second</div>
<div id="count"></div>
<button id="stop">click me</button>
```

```
let textCount = document.getElementById("count");
let btnStop = document.getElementById("stop");



function countDown() {


    if (     ) {

    }
    else
    {

    }
btnStop.addEventListener( );
}
function stopBomb()
{


}
```

setTimeout() is an asynchronous function

```
function curentDate() {
    const now = new Date();
    return now.getHours() + " : " + now.getMinutes() + ": " + now.getSeconds()
}

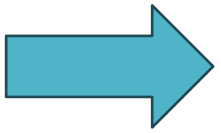
setTimeout(() => {
    console.log("this is the first message " + curentDate());
}, 5000);
setTimeout(() => {
    console.log("this is the second message " + curentDate());
}, 3000);
setTimeout(() => {
    console.log("this is the third message " + curentDate());
}, 1000);
```

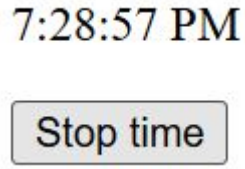
```
this is the third message 15 : 8: 3
this is the second message 15 : 8: 5
this is the first message 15 : 8: 7
```

setTimeout – setInterval()

setInterval() method continues calling the function at specified intervals until clearInterval() is called, or the window is closed

```
<p id="demo"></p>
<button onclick="myStopFunction()">Stop time</button>
```

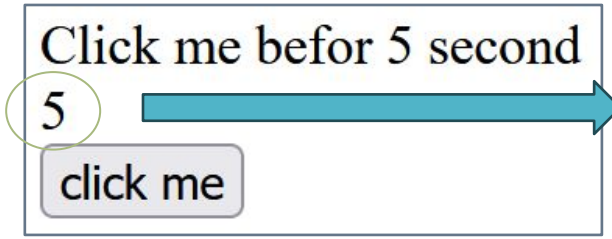





```
const myInterval = setInterval(myTimer, 1000);

function myTimer() {
  const date = new Date();
  document.getElementById("demo").innerHTML = date.toLocaleTimeString();
}

function myStopFunction() {
  clearInterval(myInterval);
```



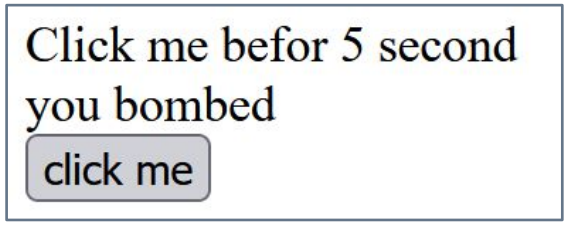
Counter count down

4 -> 3 -> 2 -> 1 -> 0

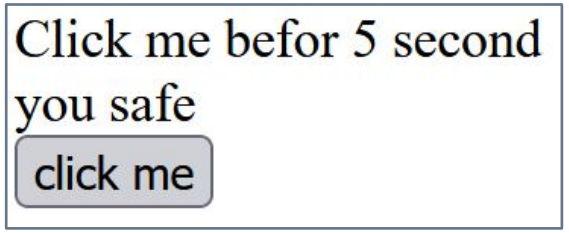
setInterval()

```
let textCount = document.getElementById("count");
let btnStop = document.getElementById("stop");
```

No click



Click intime




```
<div>Click me befor 5 second</div>
<div id="count"></div>
<button id="stop">click me</button>
```

Web API



Web API Application Programming Interface

An API is some kind of interface that includes a set of functions and subroutines that allow programmers to access specific features or data of an application, operating system or other services.

A Web API is an application programming interface for the Web

Some of the main HTML5 APIs

Geolocation API - access the current location of a user
Drag and Drop API - enables to use drag-and-drop features in browsers
Web Storage API - let browsers store key/value pairs
Web Workers API - allows a JavaScript to run in the background, without affecting the performance of the page
Server-Sent Events API - allows a web page to automatically get updates from a server

Geolocation API

The Geolocation API is used to get the user's current location

```
function success(pos) {
    const crd = pos.coords;

    console.log("Your current position is:");
    console.log(`Latitude : ${crd.latitude}`);
    console.log(`Longitude: ${crd.longitude}`);
    console.log(`More or less ${crd.accuracy} meters.`);
}
function error(err) {
    console.warn(`ERROR(${err.code}): ${err.message}`);
}
navigator.geolocation.getCurrentPosition(success, error);
```

```
Your current position is:
Latitude : 13.7563309
Longitude: 100.5017651
More or less 340764.44056023203 meters.
```

Drag and Drop API

enables an element to be dragged and dropped

Make an Element Draggable	set the `draggable` attribute to true
What to Drag	ondragstart and setData() The dragged data is the id of the dragged element
Where to Drop	Ondragover calling the preventDefault() method
Do the Drop	Append the dragged element into the drop element

By default, data/elements cannot be dropped in other elements.
To allow a drop, we must prevent the default handling of the element.

```
<div id="div1" ondrop="dropHandler(event)" ondragover="dragoverHandler(event)"></div>

<img id="img1" src="img_logo.gif" draggable="true"
ondragstart="dragstartHandler(event)" width="336" height="69">


<script>
function dragstartHandler(ev) {
  ev.dataTransfer.setData("text", ev.target.id); //value is the id of the draggable element
}

function dragoverHandler(ev) {
  ev.preventDefault();
}

function dropHandler(ev) {
  ev.preventDefault();
  const data = ev.dataTransfer.getData("text");
  ev.target.appendChild(document.getElementById(data));
}
</script>
```

https://www.w3schools.com/html/tryit.asp?filename=tryhtml5_draganddrop

Web Storage API

applications can store data locally within the user's browser

two objects for storing data in the browser

`window.localStorage`

stores data with no expiration date

data is not lost when the browser tab is closed

`window.sessionStorage`

stores data for one session

data is lost when the browser tab is closed

localStorage Object

```
// Save data to localStorage
localStorage.setItem("key", "value");

// Get saved data from localStorage
let data = localStorage.getItem("key");

// Remove saved data from localStorage
localStorage.removeItem("key");

// Remove all saved data from localStorage
localStorage.clear();
```

Test Web Storage API Support

```
const x = document.getElementById("result");
if (typeof(Storage) !== "undefined") {
  x.innerHTML = "Your browser supports Web storage!";
} else {
  x.innerHTML = "Sorry, no Web storage support!";
}
```

localStorage.setItem() - create name/value pairs

```
const x = document.getElementById("result");
if (typeof(Storage) !== "undefined") {
  // Store
  localStorage.setItem("lastname", "Smith");
  localStorage.setItem("bgcolor", "yellow");
  // Retrieve
  x.innerHTML = localStorage.getItem("lastname");
  x.style.backgroundColor = localStorage.getItem("bgcolor");
} else {
  x.innerHTML = "Sorry, no Web storage support!";
}
```



To remove the "lastname" localStorage item

```
localStorage.removeItem("lastname");
```

Storing an Object/Array

Convert the JavaScript object or array into a JSON string
Store the resulting JSON string in LocalStorage



Retrieve the JSON string from LocalStorage
Parse the JSON string back into a JavaScript object or array

```
<h2>Store and retrieve data from local storage.</h2>
<p id="demo"></p>

<script>
// Storing data:
const myObj = { name: "John", age: 31, city: "New York" };
const myJSON = JSON.stringify(myObj);
localStorage.setItem("testJSON", myJSON);

// Retrieving data:
let text = localStorage.getItem("testJSON");
let obj = JSON.parse(text);
document.getElementById("demo").innerHTML = obj.name;
</script>
```

sessionStorage Object

sessionStorage object stores the data for only one session!
The data is deleted when the user closes the specific browser tab

```
// Save data to sessionStorage
sessionStorage.setItem("key", "value");

// Get saved data from sessionStorage
let data = sessionStorage.getItem("key");

// Remove saved data from sessionStorage
sessionStorage.removeItem("key");

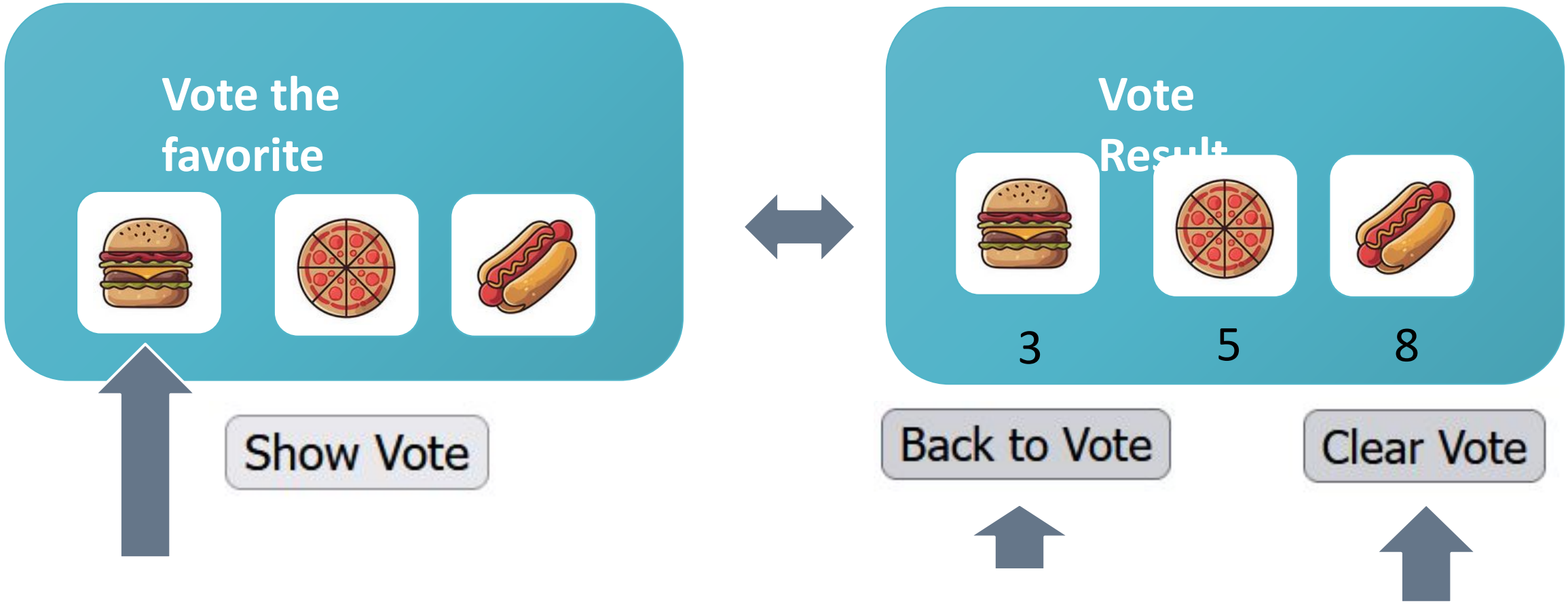
// Remove all saved data from sessionStorage
sessionStorage.clear();
```


todo

Create vote and show result

vote.htm

Showvote.htm



Click icon to vote

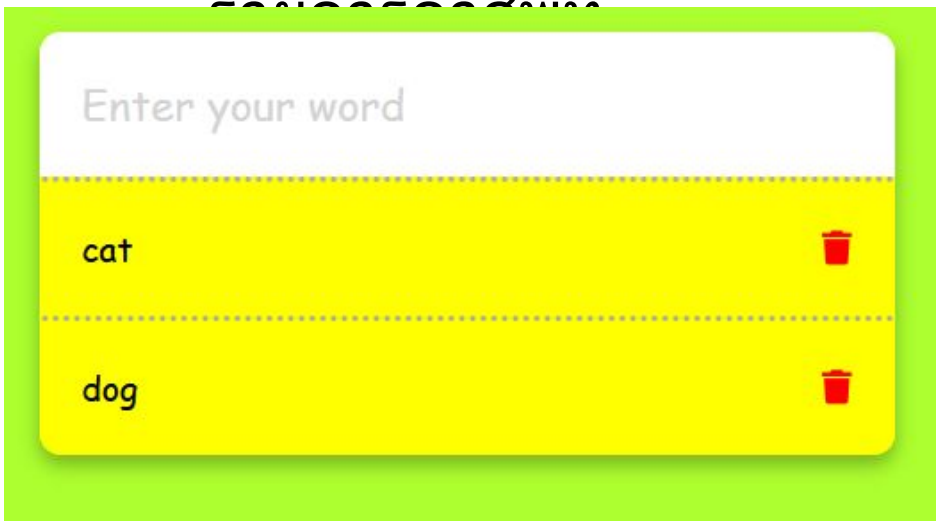
Back to vote without clear value

Clear vote to 0 0 0

เกมเรียงคำศัพท์

หน้า

มีหน้าเพิ่มลบ

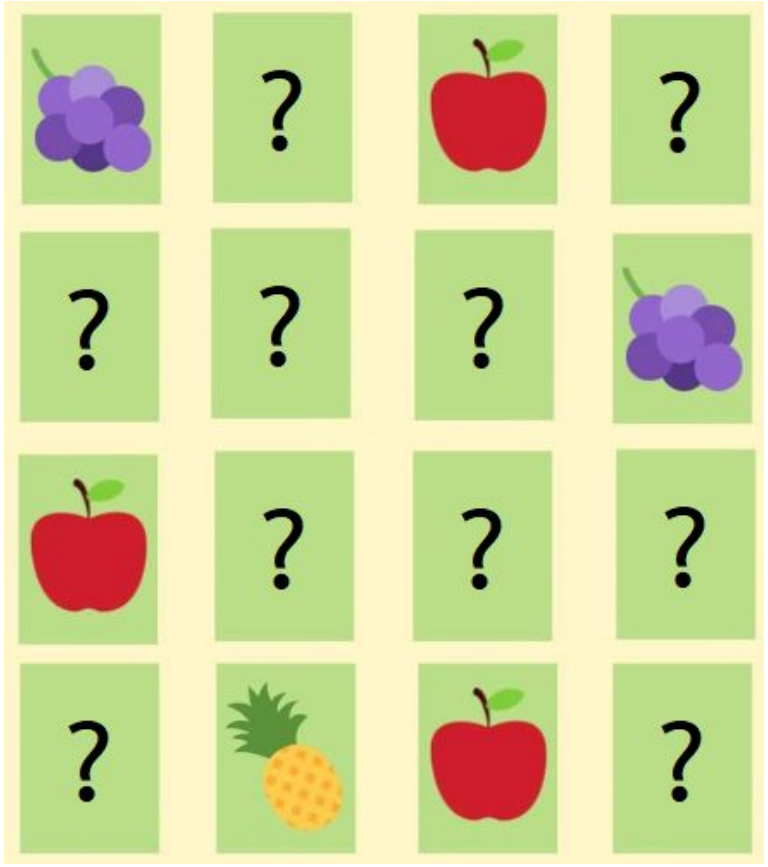


หน้า

สุ่มคำศัพท์จากรายการคำศัพท์
สุ่มสลับตำแหน่งตัวอักษรในคำศัพท์
ลากตัวอักษรไปวางตามตำแหน่ง
ตรวจสอบเมื่อวางครบทั้ง 3 ตัวอักษรแล้ว
มีการจับเวลา กำหนด timeout
วนทายครบ 5 คำสรุปคะแนน
กด reset เพื่อเริ่มเล่นนับคะแนนใหม่



เกม จับคู่



Reset

สุ่มตำแหน่งผลลัพธ์ และแสดงเป็นเครื่องหมายคำถาม

กดเปิดเครื่องหมายคำถามครั้งที่ 1 เปิดผลลัพธ์ภาพที่ 1

กดเปิดเครื่องหมายคำถามครั้งที่ 2 เปิดผลลัพธ์ภาพที่ 2
- หากไม่ตรงกันรอ 2 วินาที ปิดภาพ 1 และ 2
- หากตรงกัน เปิดค้างไว้

ปุ่ม reset สำหรับเริ่มเกมใหม่